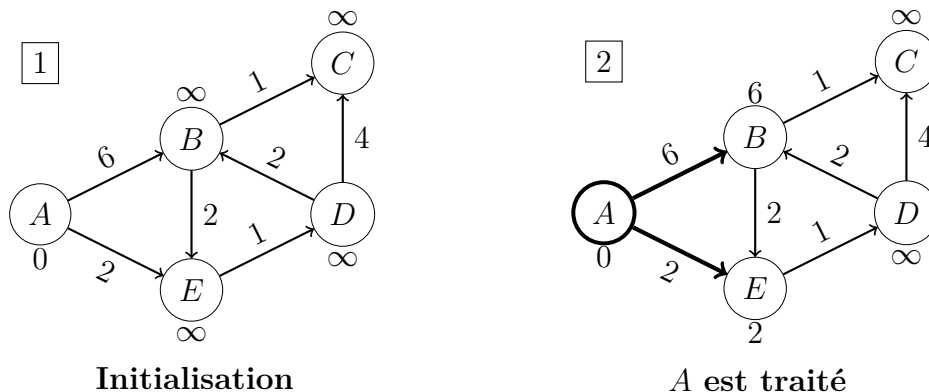


2.4.4 Algorithme de Dijkstra : Cas des valuations positives.

L'ordre dans lequel les arcs sont examinés dans l'algorithme de Ford calculant les plus courts chemins issus d'un sommet donnés, est arbitraire. Il s'en suit que, pour certains exemples, le nombre d'itérations effectuées au cours de l'algorithme peut être très élevé. Dans le cas où les valuations des arcs sont **positives**, nous allons donner un algorithme dû à **Dijkstra** qui évite cet inconvénient. Ce dernier reprend le même principe que l'algorithme de Ford, en ajoutant une règle établissant l'ordre dans lequel les arcs du graphe sont examinés. On part du sommet s pour aboutir au sommet t . Dans cet algorithme, le traitement d'un sommet i consistera à examiner successivement tous les arcs d'origine i . Tous les sommets du graphe seront traités une fois et une seule, suivant un ordre déterminé dynamiquement au cours de l'algorithme.

1. initialement $\lambda_i \leftarrow +\infty$, pour $i \neq s$; $\lambda_s \leftarrow 0$; tout sommet est « non traité »;
2. tant que tout les sommets ne sont pas traités faire
3. soit i un sommet non traité, de valeur λ_i minimale (parmi les sommets non traités)
4. pour tout arc (i, j) faire
5. si $\lambda_i + c(i, j) < \lambda_j$ alors $\lambda_j \leftarrow \lambda_i + c(i, j)$
6. le sommet i est « traité ».

Les figures ci-dessous illustrent sur un exemple le déroulement de l'algorithme. Le graphe traité est représenté en haut à gauche après la phase d'initialisation. Les plus courts chemins à déterminer ont pour origine le sommet A . Les sommets traités sont grisés alors que le sommet en cours de traitement est cerclé d'un filet épais. Les arcs modifiant les valeurs λ_i , sont représentés en gras. Sur la dernière figure (en bas) les arcs en gras sont ceux utilisés lors de la dernière modification des valeurs λ_i . En remontant ces arcs nous obtenons l'arborescence des plus courts chemins issus du sommet A .



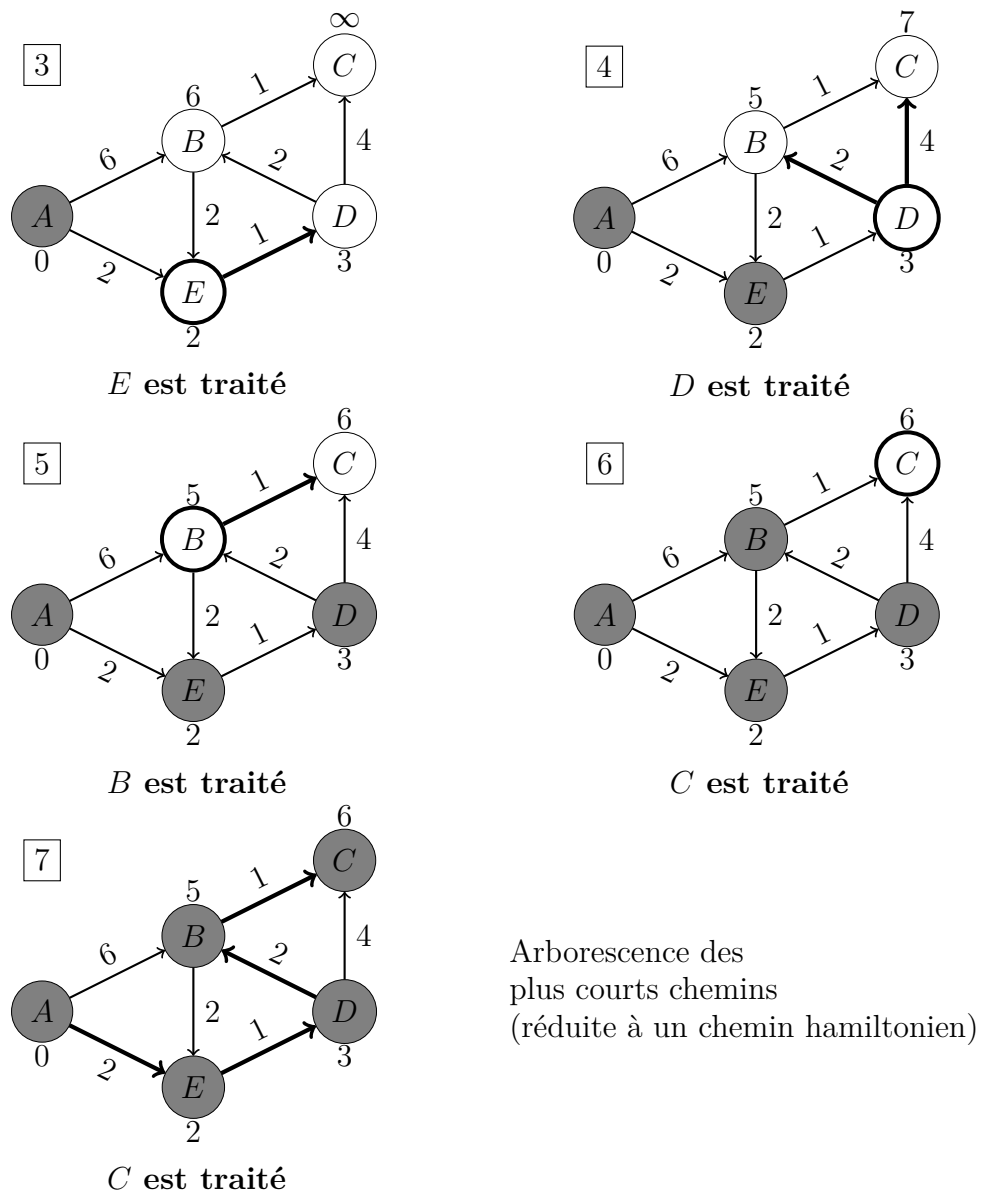


FIGURE 2.6.1 - Exemple d'exécution de l'algorithme de Dijkstra

En utilisant des structures de données appropriées, la complexité de l'algorithme le **Dijkstra** est $O(m \log n)$.

2.4.5 Algorithme de Bellman : Cas des graphes sans circuit.

Nous allons considérer ici le cas le plus simple : celui des graphes sans circuit. Nous donnons un algorithme dû à Bellman calculant les plus courts chemins à

partir d'une origine s . Reprenant le principe de l'algorithme de Ford, en ajoutant une règle supplémentaire déterminant un ordre d'examen des arcs du graphe, cet algorithme a une complexité $O(m)$. Avant de donner l'algorithme de Bellman, nous allons définir la notion d'ordre topologique dans un graphe. Un ordre topologique est une numérotation des sommets d'un graphe sans circuit satisfaisant les contraintes suivantes: si $n(i)$ est le numéro affecté à un sommet i , $n(i)$ est compris entre 1 et n ; en outre deux sommets différents reçoivent un numéro différent; et, pour tout arc (i, j) du graphe, on a : $n(i) < n(j)$. Il est aisé de démontrer qu'un graphe est sans circuit si et seulement si il admet un ordre topologique. D'autre part, des algorithmes de complexité $O(m)$ peuvent aisément être mis en œuvre pour déterminer une numérotation topologique. Dans la suite de cette partie, nous confondrons le nom et le numéro attribué à un sommet dans la numérotation topologique. De plus nous supposerons que les plus courts chemins à déterminer auront pour origine le sommet 1 et que le sommet n est une sortie.

Comme dans l'algorithme de Dijkstra, le traitement d'un sommet i consistera examiner successivement tous les arcs d'origine i . Tous les sommets du graphe seront traités une fois et une seule, suivant une numérotation topologique des sommets. L'algorithme de Bellman s'écrit de la manière suivante :

1. initialement $\lambda_i \leftarrow +\infty$, pour $i \neq 1$; $\lambda_1 \leftarrow 0$
2. pour $i = 1$ à $n - 1$ faire
3. pour tout arc (i, j) faire
4. si $\lambda_i + c(i, j) < \lambda_j$ alors $\lambda_j \leftarrow \lambda_i + c(i, j)$

La figure 4.12 illustre sur un exemple le déroulement de l'algorithme. Le graphe traité est représenté en haut à gauche, après la phase d'initialisation. Pour chaque itération, le sommet en cours de traitement est cerclé en épais et les arcs modifiant les valeurs λ_i , sont représentés en gras. Sur la figure en bas à droite, les arcs en gras sont ceux utilisés lors de la dernière modification des valeurs λ_1 . En remontant ces arcs nous obtenons un plus court chemin issu du sommet 1.

Le lecteur vérifiera ci-après la numérotation topologique des sommets: pour tout arc (i, j) , on a: $i < j$.

Remarque : Nous avons traité jusqu'à présent le cas des chemins de valeur minimale; nous allons voir que, dans quelques problèmes, ce sont, au contraire, les chemins de valeur maximale qui importent.

Il est facile de transformer certains algorithmes valables pour le premier cas en algorithmes utilisables pour le second, tel l'algorithme de Ford. Les circuits absorbants sont alors de valeur strictement positive et les tests de type: « $\lambda_i + c(i, j) < \lambda_j$ » sont remplacés par : « $\lambda_i + c(i, j) > \lambda_j$ ». (Attention: l'algorithme de Dijkstra ne s'adapte pas au cas d'une maximisation.)

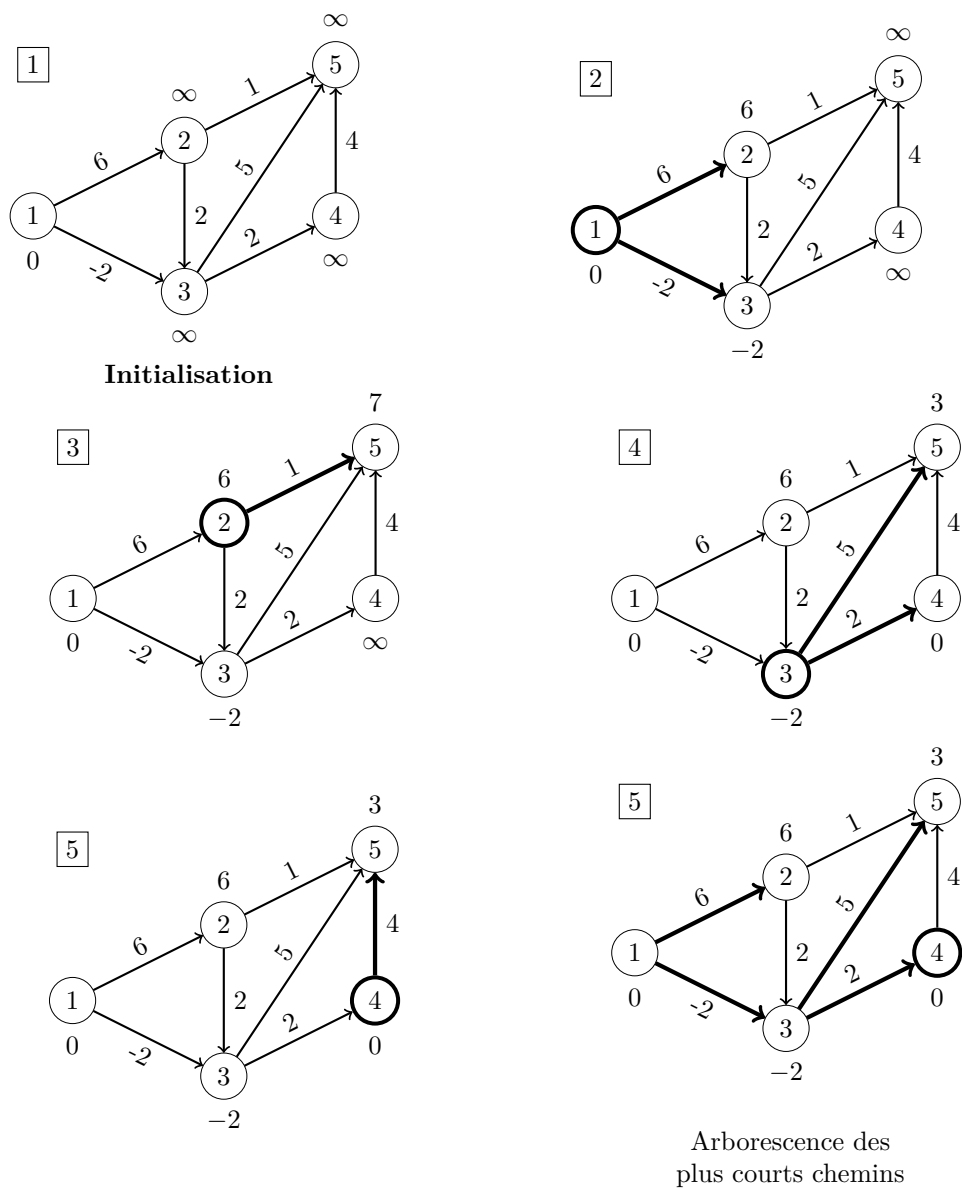


FIGURE 2.6.2 - Exemple d'exécution de l'algorithme de Bellman